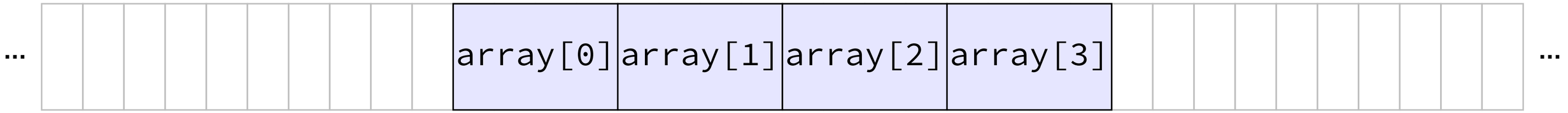
C and cache misses (1)

```
int array[4];
...
int even_sum = 0, odd_sum = 0;
even_sum += array[0];
odd_sum += array[1];
even_sum += array[2];
odd_sum += array[3];
```

Assume everything but `array` is kept in registers (and the compiler does not do anything funny).

How many data cache misses on a 1-set direct-mapped cache with 8B blocks?

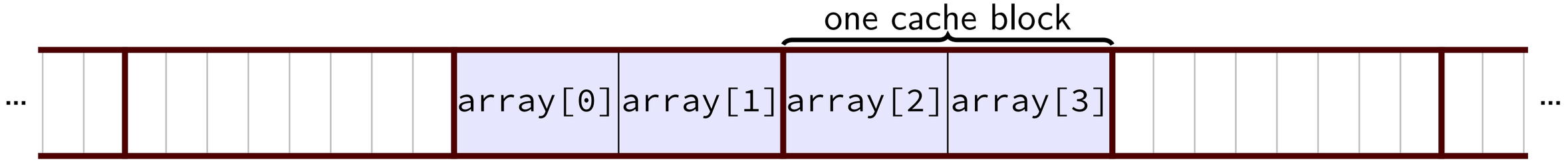
some possibilities — question



how do cache blocks correspond to array elements?

not enough information provided!

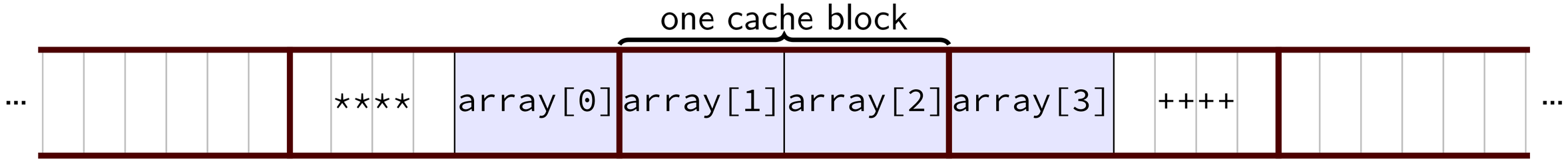
some possibilities — good case



if array[0] starts at beginning of a cache block, array split across two cache blocks

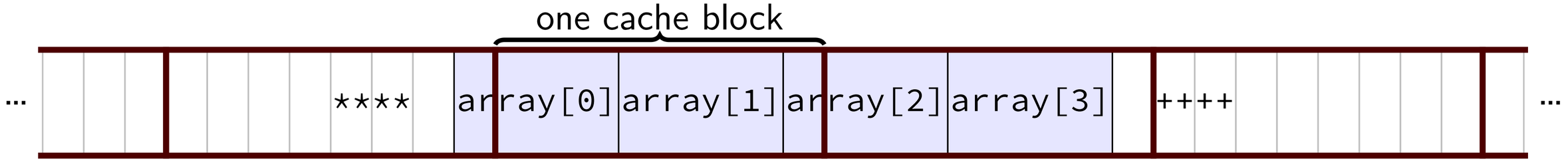
memory access	**cache contents afterwards**
read `array[0]` (miss)	`{array[0], array[1]}`
read `array[1]` (hit)	`{array[0], array[1]}`
read `array[2]` (miss)	`{array[2], array[3]}`
read `array[3]` (hit)	`{array[2], array[3]}`

some possibilities — not-so-good case



memory access	cache contents afterwards
read `array[0]` (miss)	`{****, array[1]}`
read `array[1]` (miss)	`{array[1], array[2]}`
read `array[2]` (hit)	`{array[1], array[2]}`
read `array[3]` (miss)	`{array[3], ++++}`

some possibilities — bad case



memory access	cache contents afterwards
read `array[0]` byte 0 (miss)	`{****, array[0] byte 0}`
read `array[0]` byte 1-3 (miss)	`{array[0] byte 1-3, array[1], array[2] byte 0}`
read `array[1]` (hit)	`{array[0] byte 1-3, array[1], array[2] byte 0}`
read `array[2]` byte 0 (hit)	`{array[0] byte 1-3, array[1], array[2] byte 0}`
read `array[2]` byte 1-3 (hit)	`{array[2] byte 1-3, array[3], ++++}`
read `array[3]` (hit)	`{array[2] byte 1-3, array[3], ++++}`

aside: alignment

compilers and malloc/new implementations usually try to *align* values

align = make address be multiple of something

most important reason: don't cross cache block boundaries

C and cache misses (2)

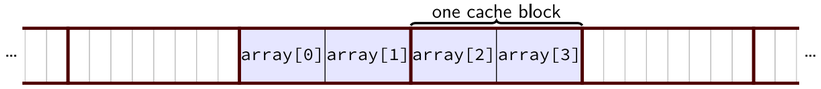
```
int array[4];
int even_sum = 0, odd_sum = 0;
even_sum += array[0];
even_sum += array[2];
odd_sum += array[1];
odd_sum += array[3];
```

Assume everything but `array` is kept in registers (and the compiler does not do anything funny).

Assume `array[0]` at beginning of cache block.

How many data cache misses on a 1-set direct-mapped cache with 8B blocks?

exercise solution



if array[0] starts at beginning of a cache block, array split across two cache blocks

memory access	cache contents afterwards
read `array[0]` (miss)	`{array[0], array[1]}`
read `array[2]` (miss)	`{array[2], array[3]}`
read `array[1]` (miss)	`{array[0], array[1]}`
read `array[3]` (miss)	`{array[2], array[3]}`

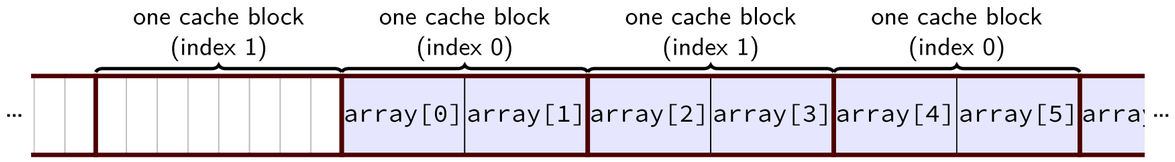
C and cache misses (3)

```
int array[8]; /* assume aligned */
...
int even_sum = 0, odd_sum = 0;
even_sum += array[0];
even_sum += array[2];
even_sum += array[4];
even_sum += array[6];
odd_sum += array[1];
odd_sum += array[3];
odd_sum += array[5];
odd_sum += array[7];
```

Assume everything but `array` is kept in registers (and the compiler does not do anything funny).

How many data cache misses on a 2-set direct-mapped cache with 8B blocks?

assigning blocks to sets



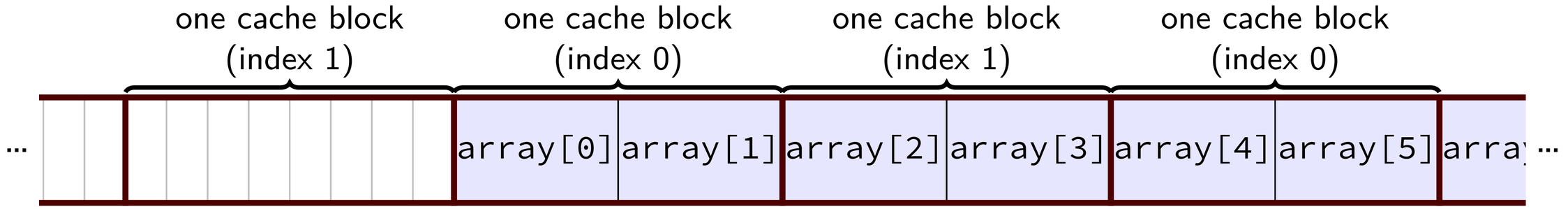
block of memory maps to set 0, then set 1, then set 0, then set 1

- if more sets in cache would be: set 0, set 1, set 2, … set N, set 0, set 1, …

could be that array[0] is in set 1, not 0

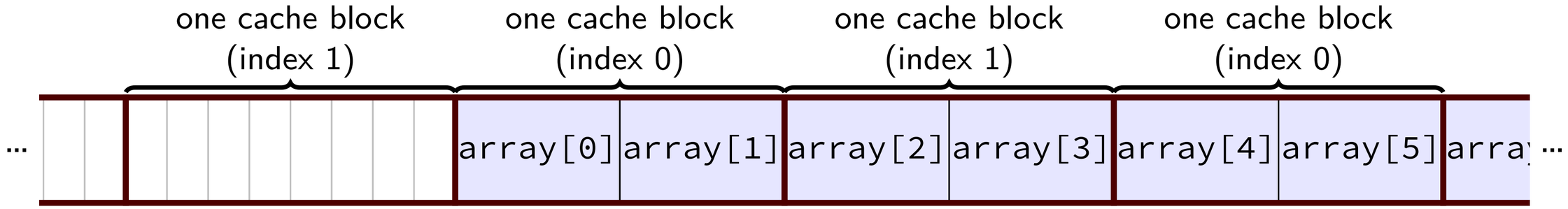
- would just swap sets 0 and 1 — same accesses otherwise
- no change to miss rate

exercise solution



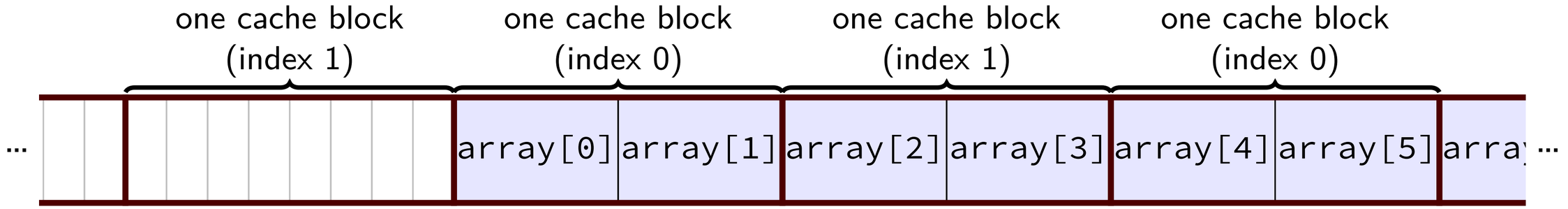
memory access	set 0 aftewards	set 1 afterwards
—	(empty)	(empty)
read `array[0]` (miss)	`{array[0], array[1]}`	(empty)
read `array[2]` (miss)	`{array[0], array[1]}`	`{array[2], array[3]}`
read `array[4]` (miss)	`{array[4], array[5]}`	`{array[2], array[3]}`
read `array[6]` (miss)	`{array[4], array[5]}`	`{array[6], array[7]}`
read `array[1]` (miss)	`{array[0], array[1]}`	`{array[6], array[7]}`
read `array[3]` (miss)	`{array[0], array[1]}`	`{array[2], array[3]}`
read `array[5]` (miss)	`{array[4], array[5]}`	`{array[2], array[3]}`
read `array[7]` (miss)	`{array[4], array[5]}`	`{array[6], array[7]}`

exercise solution



memory access	set 0 aftewards	set 1 afterwards
—	(empty)	(empty)
read `array[0]` (miss)	`{array[0], array[1]}`	(empty)
read `array[2]` (miss)	`{array[0], array[1]}`	`{array[2], array[3]}`
read `array[4]` (miss)	`{array[4], array[5]}`	`{array[2], array[3]}`
read `array[6]` (miss)	`{array[4], array[5]}`	`{array[6], array[7]}`
read `array[1]` (miss)	`{array[0], array[1]}`	`{array[6], array[7]}`
read `array[3]` (miss)	`{array[0], array[1]}`	`{array[2], array[3]}`
read `array[5]` (miss)	`{array[4], array[5]}`	`{array[2], array[3]}`
read `array[7]` (miss)	`{array[4], array[5]}`	`{array[6], array[7]}`

exercise solution



memory access	set 0 aftewards	set 1 afterwards
—	(empty)	(empty)
read `array[0]` (miss)	{`array[0]`, `array[1]`}	(empty)
read `array[2]` (miss)	{`array[0]`, `array[1]`}	{`array[2]`, `array[3]`}
read `array[4]` (miss)	{`array[4]`, `array[5]`}	{`array[2]`, `array[3]`}
read `array[6]` (miss)	{`array[4]`, `array[5]`}	{`array[6]`, `array[7]`}
read `array[1]` (miss)	{`array[0]`, `array[1]`}	{`array[6]`, `array[7]`}
read `array[3]` (miss)	{`array[0]`, `array[1]`}	{`array[2]`, `array[3]`}
read `array[5]` (miss)	{`array[4]`, `array[5]`}	{`array[2]`, `array[3]`}
read `array[7]` (miss)	{`array[4]`, `array[5]`}	{`array[6]`, `array[7]`}

C and cache misses (4)

```
int array[8]; /* assume aligned */
...
int even_sum = 0, odd_sum = 0;
even_sum += array[0];
odd_sum += array[3];
even_sum += array[6];
odd_sum += array[1];
even_sum += array[4];
odd_sum += array[7];
even_sum += array[2];
odd_sum += array[5];
```

Assume everything but `array` is kept in registers (and the compiler does not do anything funny).

How many data cache misses on a 2-set direct-mapped cache with 8B blocks?

```
int array[8]; /* assume aligned */
...
int even_sum = 0, odd_sum = 0;
even_sum += array[0];
odd_sum += array[3];
even_sum += array[6];
odd_sum += array[1];
even_sum += array[4];
odd_sum += array[7];
even_sum += array[2];
odd_sum += array[5];
```

C and cache misses (5)

```
int array[1024]; /* assume aligned */ int even = 0, odd = 0;
even += array[0];
even += array[2];
even += array[512];
even += array[514];
odd += array[1];
odd += array[3];
odd += array[511];
odd += array[513];
```

Assume everything but `array` is kept in registers (and the compiler does not do anything funny).
observation: `array[0]` and `array[512]` exactly 2KB apart

How many data cache misses on a 2KB direct mapped cache with 16B blocks?

C and cache misses (6)

```
int array[1024]; /* assume aligned */ int even = 0, odd = 0;
even += array[0];
even += array[2];
even += array[500];
even += array[502];
odd += array[1];
odd += array[3];
odd += array[501];
odd += array[503];
```

Assume everything but `array` is kept in registers (and the compiler does not do anything funny).

How many data cache misses on a 2KB direct mapped cache with 16B blocks?

misses with skipping

```
int array1[512]; int array2[512];
...
for (int i = 0; i < 512; i += 1)
    sum += array1[i] * array2[i];
}
```

Assume everything but `array1`, `array2` is kept in registers (and the compiler does not do anything funny).

About how many *data cache misses* on a 2KB direct-mapped cache with 16B cache blocks?

Hint: depends on relative placement of `array1`, `array2`

best/worst case

`array1[i]` and `array2[i]` always different sets:

= distance from `array1` to `array2` not multiple of `#sets` × `bytes/set`

2 misses every 4 `i`

blocks of 4 `array1[X]` values loaded, then used 4 times before loading next block

(and same for `array2[X]`)

`array1[i]` and `array2[i]` same sets:

= distance from `array1` to `array2` is multiple of `#sets` × `bytes/set`

2 misses every `i`

block of 4 `array1[X]` values loaded, one value used from it,

then, block of 4 `array2[X]` values replaces it, one value used from it, ...

worst case in practice?

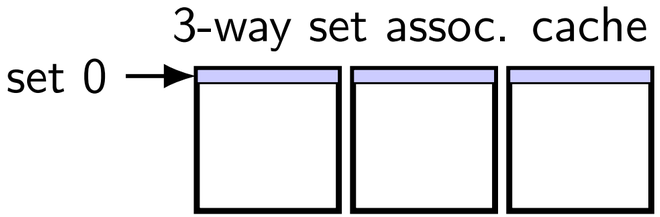
can have worst spacing happen by accident:

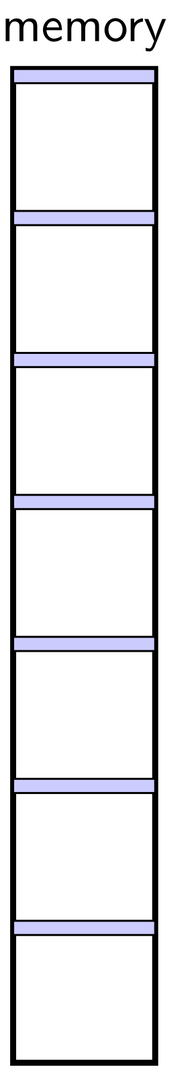
two structs/arrays placed at start of newly allocated page

- worst-case spacing likely small multiple of page size

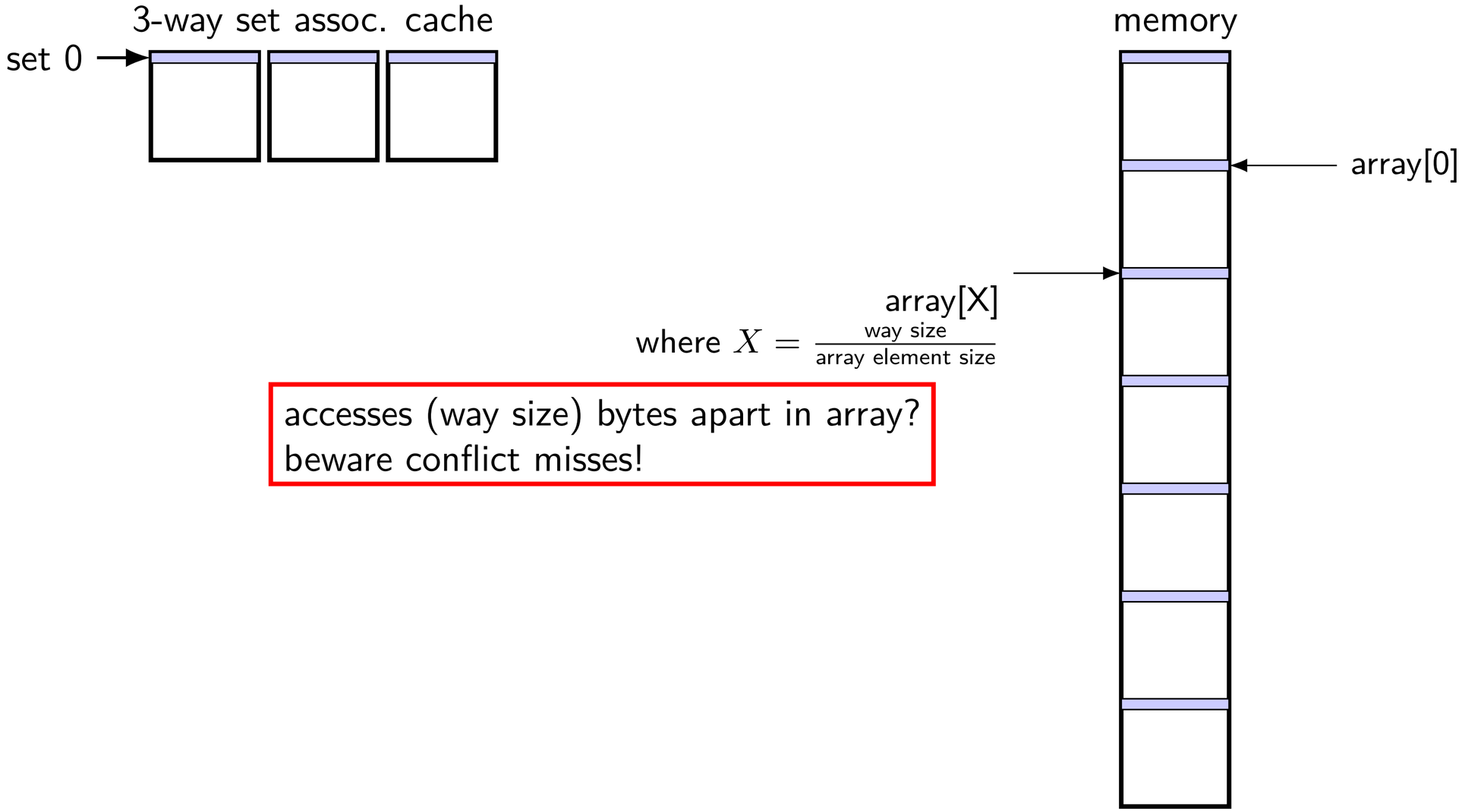
columns of matrix with power-of-two width

mapping of sets to memory (3-way)





mapping of sets to memory (3-way)



accesses (way size) bytes apart in array?
beware conflict misses!

C and cache misses (assoc)

```
int array[1024]; /* assume aligned */
int even_sum = 0, odd_sum = 0;
even_sum += array[0];
even_sum += array[2];
even_sum += array[512];
even_sum += array[514];
odd_sum += array[1];
odd_sum += array[3];
odd_sum += array[511];
odd_sum += array[513];
```

Assume everything but `array` is kept in registers (and the compiler does not do anything funny).
observation: `array[0]`, `array[256]`, `array[512]`, `array[768]` in same set

How many data cache misses on a 2KB 2-way **set associative** cache with 16B blocks?

C and cache misses (assoc)

```
int array[1024]; /* assume aligned */
int even_sum = 0, odd_sum = 0;
even_sum += array[0];
even_sum += array[256];
even_sum += array[512];
even_sum += array[768];
odd_sum += array[1];
odd_sum += array[257];
odd_sum += array[513];
odd_sum += array[769];
```

Assume everything but `array` is kept in registers (and the compiler does not do anything funny).
observation: `array[0]`, `array[256]`, `array[512]`, `array[768]` in same set

How many data cache misses on a 2KB 2-way set associative cache with 16B blocks?

C and cache misses (struct)

```
typedef struct {
    int a_value, b_value;
    int other_values[10];
} item;
item items[5];
int a_sum = 0, b_sum = 0;
for (int i = 0; i < 5; ++i)
    a_sum += items[i].a_value;
for (int i = 0; i < 5; ++i)
    b_sum += items[i].b_value;
```

observation: 12 ints in struct: only first two used

equivalent to accessing array[0], array[12], array[24], etc.

… then accessing array[1], array[13], array[25], etc.

C and cache misses (struct, rewritten?)

```
typedef struct {
    int a_value, b_value;
    int other_values[10];
} item;
item items[5];
int a_sum = 0, b_sum = 0;
for (int i = 0; i < 5; ++i)
    a_sum += items[i].a_value;
for (int i = 0; i < 5; ++i)
    b_sum += items[i].b_value;
```

```
int array[60];
int a_sum = 0, b_sum = 0;
for (int i = 0; i < 60; i += 12)
    a_sum += array[i];
for (int i = 1; i < 60; i += 12)
    b_sum += array[i];
```

C and cache misses (struct, rewritten?)

```
int array[60];
int a_sum = 0, b_sum = 0;
for (int i = 0; i < 60; i += 12)
    a_sum += array[i];
for (int i = 1; i < 60; i += 12)
    b_sum += array[i];
```

Assume everything but `array` is kept in registers (and the compiler does not do anything funny) and array at beginning of cache block.

How many *data cache misses* on a 64B direct-mapped cache with 16B cache blocks?

observation 1: first loop has 5 misses — first accesses to each block

observation 2: array[0] and array[1], array[12] and array[13], etc. in same cache block

C and cache misses (struct, solution setup 1)

array[0 to 3]	set 0
array[4 to 7]	set 1
array[8 to 11]	set 2
array[12 to 15]	set 3
array[16 to 19]	set 0
array[20 to 23]	set 1
array[20 to 23]	set 1
…	…

C and cache misses (struct, solution setup)

access	set	block loaded (if needed)
array[0]	0	array[0 to 3]
array[12]	3	array[12 to 15]
array[24]	2	array[24 to 27]
array[36]	1	array[36 to 39]
array[48]	0	array[48 to 51]
array[1]	0	array[0 to 3]
array[13]	3	array[12 to 15]
array[25]	2	array[24 to 27]
array[37]	1	array[36 to 39]
array[49]	0	array[48 to 51]

C and cache misses (struct, solution)

access	set	block loaded (if needed)	result
array[0]	0	array[0 to 3]	miss
array[12]	3	array[12 to 15]	miss
array[24]	2	array[24 to 27]	miss
array[36]	1	array[36 to 39]	miss
array[48]	0	array[48 to 51]	miss
array[1]	0	array[0 to 3]	miss*
array[13]	3	array[12 to 15]	hit
array[25]	2	array[24 to 27]	hit
array[37]	1	array[36 to 39]	hit
array[49]	0	array[48 to 51]	miss*

C and cache misses (struct, solution written)

ints 4 byte $\rightarrow$ array[0 to 3] and array[16 to 19] in same cache set

- 64B = 16 ints stored per way
- 4 sets total

accessing array indices 0, 12, 24, 36, 48, 1, 13, 25, 37, 49

- 0, 12, 24, 36, 48 all misses

0 (set 0, array[0 to 3]), 12 (set 3, 12-15), 24 (set 2, 24-27), 36 (set 1, 36-39), 48 (set 0, 48-51)

- 13, 25, 37 will be hits (rest of block already loaded)
- 1 will be a miss (0-3 was replaced with 48-51 when 48 accessed)
- 49 will be a miss (48-51 was replaced with 0-3 when 1 accessed)

backup slides

cache accesses and C code (1)

```
int scaleFactor;

int scaleByFactor(int value) {
    return value * scaleFactor;
}
```

```
scaleByFactor:
    movl scaleFactor, %eax
    imull %edi, %eax
    ret
```

---

exericse: what data cache accesses does this function do?

- 4-byte read of scaleFactor
- 8-byte read of return address

possible scaleFactor use

```
for (int i = 0; i < size; ++i) {
    array[i] = scaleByFactor(array[i]);
}
```

misses and code (2)

```
scaleByFactor:
    movl scaleFactor, %eax
    imull %edi, %eax
    ret
```

suppose each time this is called in the loop:

return address located at address `0x7fffffe43b8`

scaleFactor located at address `0x6bc3a0`

with direct-mapped 32KB cache w/64 B blocks, what is their:

	return address	scaleFactor
tag		
index		
offset		

misses and code (2)

```
scaleByFactor:
    movl scaleFactor, %eax
    imull %edi, %eax
    ret
```

suppose each time this is called in the loop:

- return address located at address `0x7fffffffe43b8`
- scaleFactor located at address `0x6bc3a0`

with direct-mapped 32KB cache w/64 B blocks, what is their:

	return address	scaleFactor
tag	0xffffffffc	0xd7
index	*0x10e*	*0x10e*
offset	0x38	0x20

conflict miss coincidences?

obviously I set that up to have the same index

- have to use exactly the right amount of stack space…

but one of the reasons we'll want something better than direct-mapped cache

C and cache misses (warmup 3)

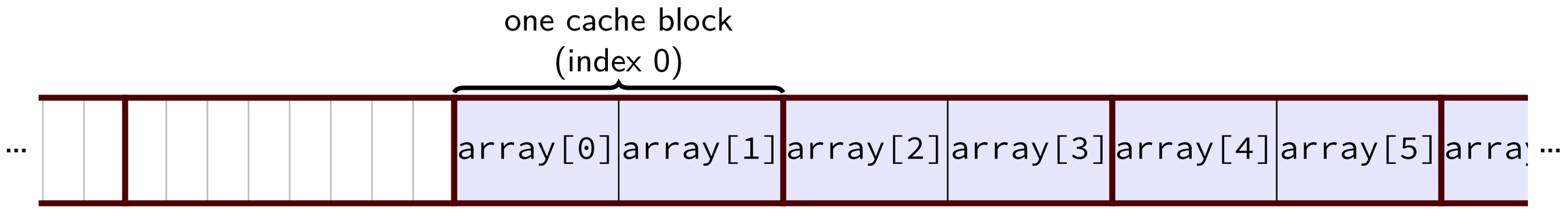
```
int array[8];
...
int even_sum = 0, odd_sum = 0;
even_sum += array[0];
odd_sum += array[1];
even_sum += array[2];
odd_sum += array[3];
even_sum += array[4];
odd_sum += array[5];
even_sum += array[6];
odd_sum += array[7];
```

Assume everything but `array` is kept in registers (and the compiler does not do anything funny), and `array[0]` at beginning of cache block.

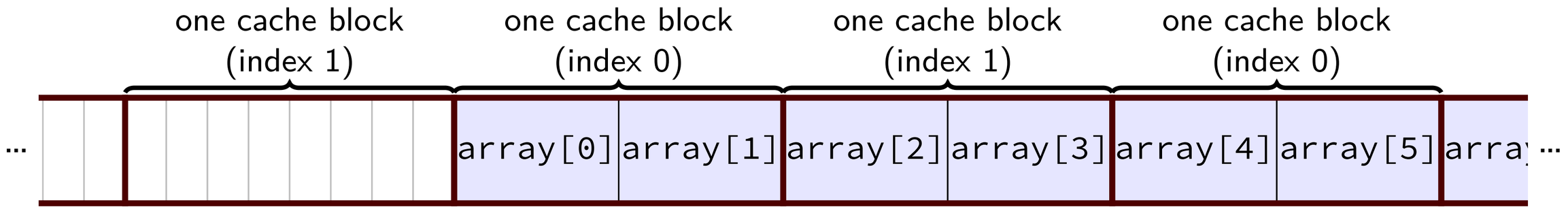
How many data cache misses on a 2-set direct-mapped cache with 8B blocks?

exercise solution

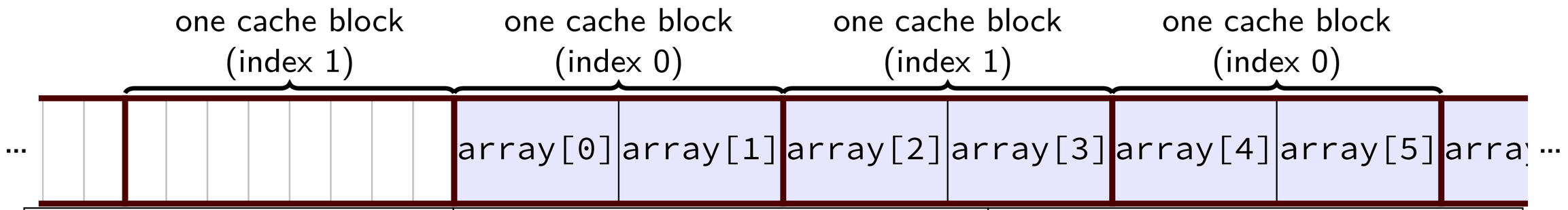
exercise solution



exercise solution

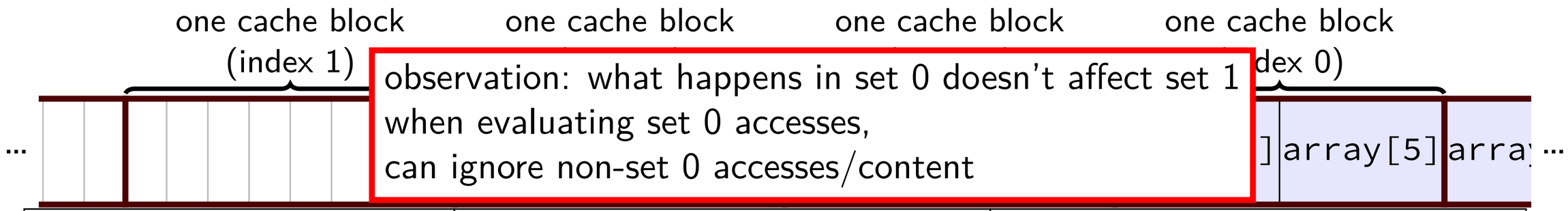


exercise solution



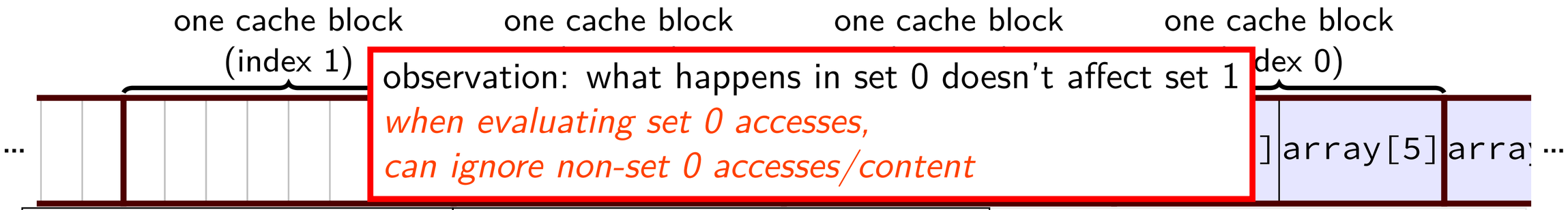
memory access	**set 0 afterwards**	**set 1 afterwards**
—	(empty)	(empty)
read array[0] (miss)	{array[0], array[1]}	(empty)
read array[1] (hit)	{array[0], array[1]}	(empty)
read array[2] (miss)	{array[0], array[1]}	{array[2], array[3]}
read array[3] (hit)	{array[0], array[1]}	{array[2], array[3]}
read array[4] (miss)	{array[4], array[5]}	{array[2], array[3]}
read array[5] (hit)	{array[4], array[5]}	{array[2], array[3]}
read array[6] (miss)	{array[4], array[5]}	{array[6], array[7]}
read array[7] (hit)	{array[4], array[5]}	{array[6], array[7]}

exercise solution



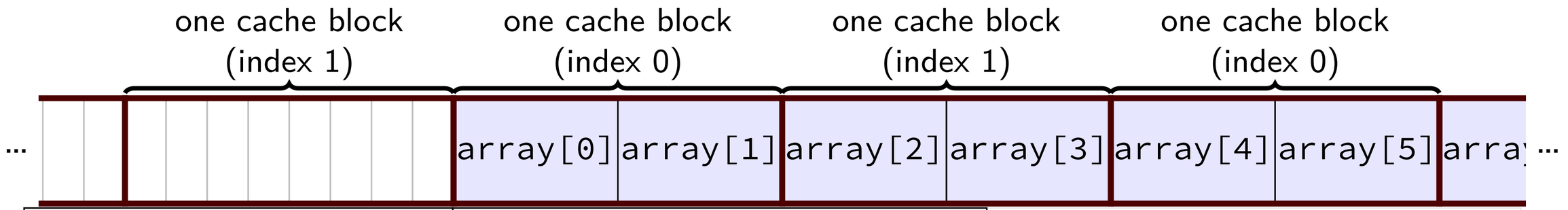
memory access	**set 0 afterwards**	**set 1 afterwards**
—	(empty)	(empty)
read array[0] (miss)	{array[0], array[1]}	(empty)
read array[1] (hit)	{array[0], array[1]}	(empty)
read array[2] (miss)	{array[0], array[1]}	{array[2], array[3]}
read array[3] (hit)	{array[0], array[1]}	{array[2], array[3]}
read array[4] (miss)	{array[4], array[5]}	{array[2], array[3]}
read array[5] (hit)	{array[4], array[5]}	{array[2], array[3]}
read array[6] (miss)	{array[4], array[5]}	{array[6], array[7]}
read array[7] (hit)	{array[4], array[5]}	{array[6], array[7]}

exercise solution



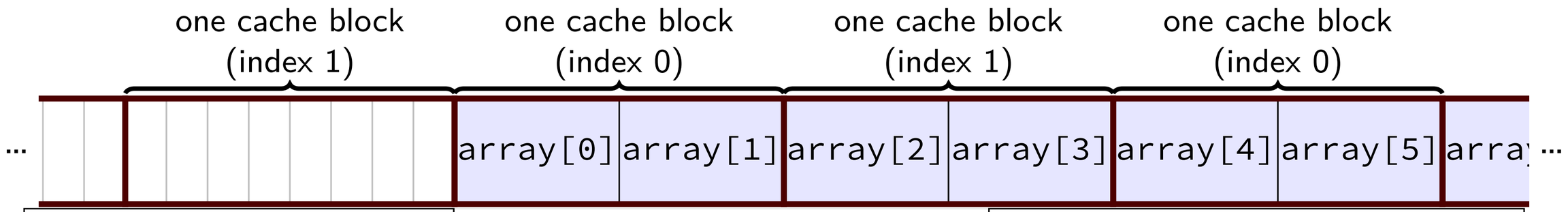
memory access	set 0 afterwards	set 1 afterwards
—	(empty)	(empty)
read array[0] (miss)	{array[0], array[1]}	(empty)
read array[1] (hit)	{array[0], array[1]}	(empty)
read array[2] (miss)	{array[0], array[1]}	{array[2], array[3]}
read array[3] (hit)	{array[0], array[1]}	{array[2], array[3]}
read array[4] (miss)	{array[4], array[5]}	{array[2], array[3]}
read array[5] (hit)	{array[4], array[5]}	{array[2], array[3]}
read array[6] (miss)	{array[4], array[5]}	{array[6], array[7]}
read array[7] (hit)	{array[4], array[5]}	{array[6], array[7]}

exercise solution



memory access	set 0 afterwards	set 1 afterwards
—	(empty)	(empty)
read array[0] (miss)	{array[0], array[1]}	(empty)
read array[1] (hit)	{array[0], array[1]}	(empty)
read array[2] (miss)	{array[0], array[1]}	{array[2], array[3]}
read array[3] (hit)	{array[0], array[1]}	{array[2], array[3]}
read array[4] (miss)	{array[4], array[5]}	{array[2], array[3]}
read array[5] (hit)	{array[4], array[5]}	{array[2], array[3]}
read array[6] (miss)	{array[4], array[5]}	{array[6], array[7]}
read array[7] (hit)	{array[4], array[5]}	{array[6], array[7]}

exercise solution



memory access	set 0 afterwards	set 1 afterwards
—	(empty)	(empty)
read array[0] (miss)	{array[0], array[1]}	(empty)
read array[1] (hit)	{array[0], array[1]}	(empty)
read array[2] (miss)	{array[0], array[1]}	{array[2], array[3]}
read array[3] (hit)	{array[0], array[1]}	{array[2], array[3]}
read array[4] (miss)	{array[4], array[5]}	{array[2], array[3]}
read array[5] (hit)	{array[4], array[5]}	{array[2], array[3]}
read array[6] (miss)	{array[4], array[5]}	{array[6], array[7]}
read array[7] (hit)	{array[4], array[5]}	{array[6], array[7]}

arrays and cache misses (1)

```
int array[1024]; // 4KB array
int even_sum = 0, odd_sum = 0;
for (int i = 0; i < 1024; i += 2) {
    even_sum += array[i + 0];
    odd_sum +=  array[i + 1];
}
```

Assume everything but `array` is kept in registers (and the compiler does not do anything funny).

How many *data cache misses* on initially empty 2KB direct-mapped cache with 16B cache blocks?

arrays and cache misses (2)

```
int array[1024]; // 4KB array
int even_sum = 0, odd_sum = 0;
for (int i = 0; i < 1024; i += 2)
    even_sum += array[i + 0];
for (int i = 0; i < 1024; i += 2)
    odd_sum +=  array[i + 1];
```

Assume everything but `array` is kept in registers (and the compiler does not do anything funny).

How many *data cache misses* on initially empty 2KB direct-mapped cache with 16B cache blocks?

explanation

2-way, 2KB set associative cache, 16B blocks

4 offset bits, 6 index bits

so addresses multiples $2^1 0$ bytes apart differ only in tag bits

example: array[0$\rightarrow{}3], $array[256\rightarrow{}259]$, $array[512\rightarrow{}515]$, $array[768\rightarrow{}$771]

those all use the same set

but sets only holds 2 things

all misses

arrays and cache misses (2b)

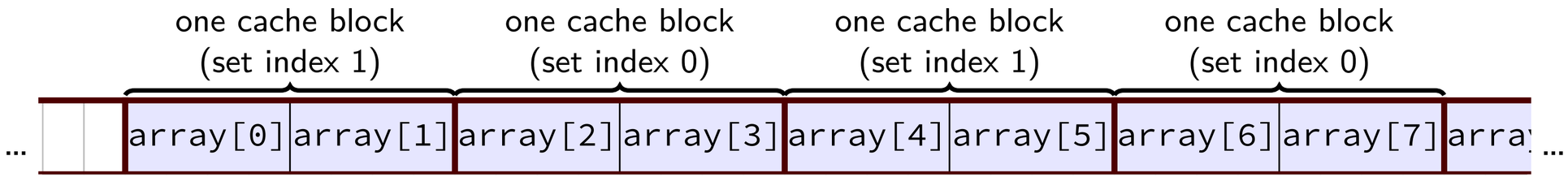
```
int array[1024]; // 4KB array
int even_sum = 0, odd_sum = 0;
for (int i = 0; i < 1024; i += 2)
    even_sum += array[i + 0];
for (int i = 0; i < 1024; i += 2)
    odd_sum +=  array[i + 1];
```

Assume everything but `array` is kept in registers (and the compiler does not do anything funny).

How many *data cache misses* on initially empty *4KB* direct-mapped cache with 16B cache blocks?

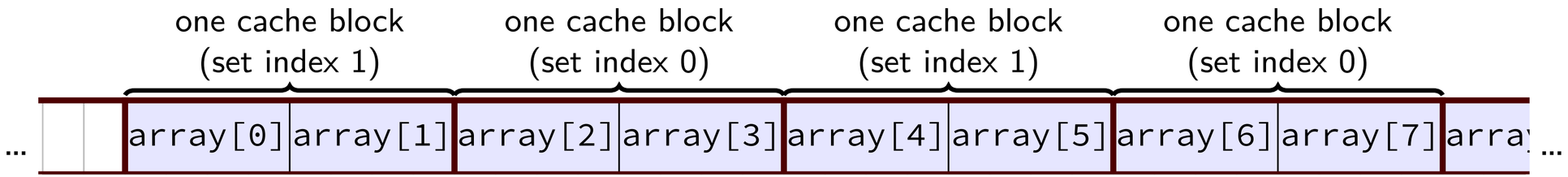
quiz exercise solution

quiz exercise solution



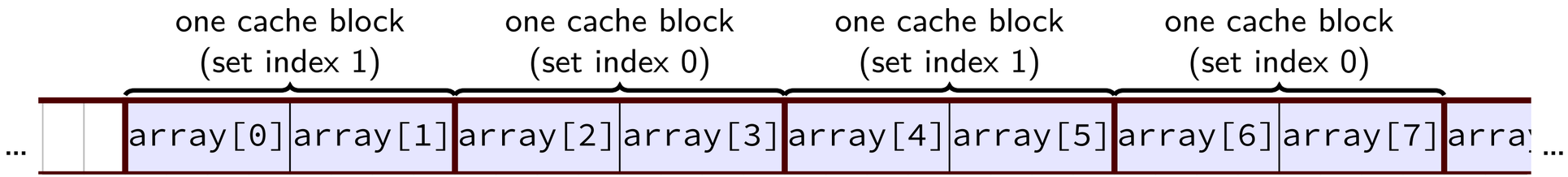
memory access	set 0 afterwards	set 1 afterwards
—	(empty)	(empty)
read array[0] (miss)	{array[0], array[1]}	(empty)
read array[3] (miss)	{array[0], array[1]}	{array[2], array[3]}
read array[6] (miss)	{array[0], array[1]}	{array[6], array[7]}
read array[1] (hit)	{array[0], array[1]}	{array[6], array[7]}
read array[4] (miss)	{array[4], array[5]}	{array[6], array[7]}
read array[7] (hit)	{array[4], array[5]}	{array[6], array[7]}
read array[2] (miss)	{array[4], array[5]}	{array[2], array[3]}
read array[5] (hit)	{array[4], array[5]}	{array[6], array[7]}
read array[8] (miss)	{array[8], array[9]}	{array[6], array[7]}

quiz exercise solution



memory access	set 0 afterwards	set 1 afterwards
—	(empty)	(empty)
read array[0] (miss)	{array[0], array[1]}	(empty)
read array[3] (miss)	{array[0], array[1]}	{array[2], array[3]}
read array[6] (miss)	{array[0], array[1]}	{array[6], array[7]}
read array[1] (hit)	{array[0], array[1]}	{array[6], array[7]}
read array[4] (miss)	{array[4], array[5]}	{array[6], array[7]}
read array[7] (hit)	{array[4], array[5]}	{array[6], array[7]}
read array[2] (miss)	{array[4], array[5]}	{array[2], array[3]}
read array[5] (hit)	{array[4], array[5]}	{array[6], array[7]}
read array[8] (miss)	{array[8], array[9]}	{array[6], array[7]}

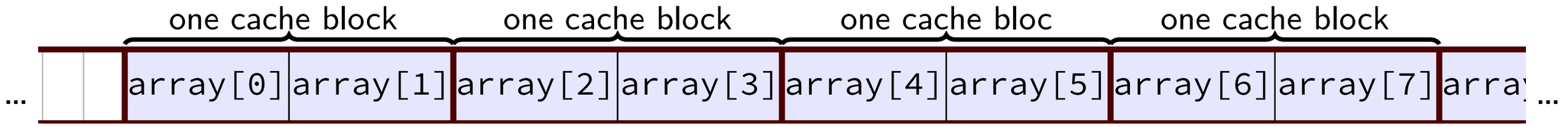
quiz exercise solution



memory access	set 0 afterwards	set 1 afterwards
—	(empty)	(empty)
read array[0] (miss)	{array[0], array[1]}	(empty)
read array[3] (miss)	{array[0], array[1]}	{array[2], array[3]}
read array[6] (miss)	{array[0], array[1]}	{array[6], array[7]}
read array[1] (hit)	{array[0], array[1]}	{array[6], array[7]}
read array[4] (miss)	{array[4], array[5]}	{array[6], array[7]}
read array[7] (hit)	{array[4], array[5]}	{array[6], array[7]}
read array[2] (miss)	{array[4], array[5]}	{array[2], array[3]}
read array[5] (hit)	{array[4], array[5]}	{array[6], array[7]}
read array[8] (miss)	{array[8], array[9]}	{array[6], array[7]}

not the quiz problem

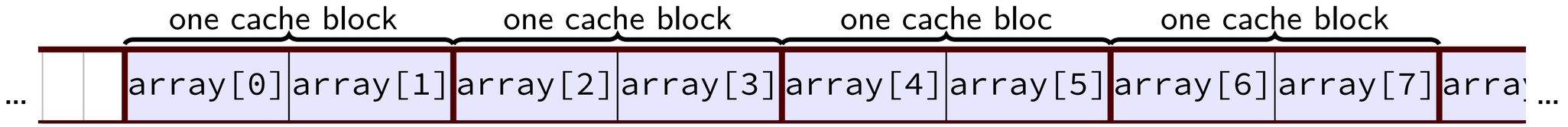
not the quiz problem



if 1-set 2-way cache instead of 2-set 1-way cache:

memory access	**single set with 2-ways, LRU first**
—	`---, ---`
read `array[0]` (miss)	`---, {array[0], array[1]}`
read `array[3]` (miss)	`{array[0], array[1]}, {array[2], array[3]}`
read `array[6]` (miss)	`{array[2], array[3]}, {array[6], array[7]}`
read `array[1]` (miss)	`{array[6], array[7]}, {array[0], array[1]}`
read `array[4]` (miss)	`{array[0], array[1]}, {array[3], array[4]}`
read `array[7]` (miss)	`{array[3], array[4]}, {array[6], array[7]}`
read `array[2]` (miss)	`{array[6], array[7]}, {array[2], array[3]}`
read `array[5]` (miss)	`{array[2], array[3]}, {array[5], array[6]}`
read `array[8]` (miss)	`{array[5], array[6]}, {array[8], array[9]}`

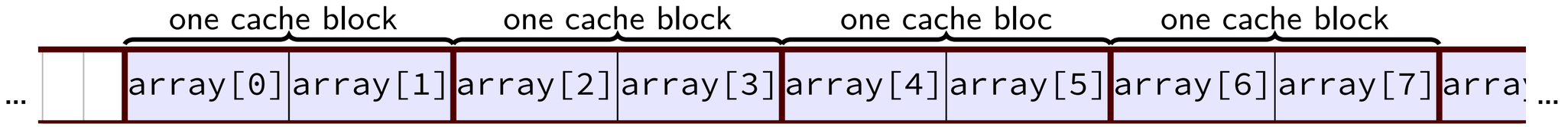
not the quiz problem



if 1-set 2-way cache instead of 2-set 1-way cache:

memory access	**single set with 2-ways, LRU first**
—	`---, ---`
read `array[0]` (miss)	`---, {array[0], array[1]}`
read `array[3]` (miss)	`{array[0], array[1]}, {array[2], array[3]}`
read `array[6]` (miss)	`{array[2], array[3]}, {array[6], array[7]}`
read `array[1]` (miss)	`{array[6], array[7]}, {array[0], array[1]}`
read `array[4]` (miss)	`{array[0], array[1]}, {array[3], array[4]}`
read `array[7]` (miss)	`{array[3], array[4]}, {array[6], array[7]}`
read `array[2]` (miss)	`{array[6], array[7]}, {array[2], array[3]}`
read `array[5]` (miss)	`{array[2], array[3]}, {array[5], array[6]}`
read `array[8]` (miss)	`{array[5], array[6]}, {array[8], array[9]}`

not the quiz problem



if 1-set 2-way cache instead of 2-set 1-way cache:

memory access	**single set with 2-ways, LRU first**
—	`---, ---`
read `array[0]` (miss)	`---, {array[0], array[1]}`
read `array[3]` (miss)	`{array[0], array[1]}, {array[2], array[3]}`
read `array[6]` (miss)	`{array[2], array[3]}, {array[6], array[7]}`
read `array[1]` (miss)	`{array[6], array[7]}, {array[0], array[1]}`
read `array[4]` (miss)	`{array[0], array[1]}, {array[3], array[4]}`
read `array[7]` (miss)	`{array[3], array[4]}, {array[6], array[7]}`
read `array[2]` (miss)	`{array[6], array[7]}, {array[2], array[3]}`
read `array[5]` (miss)	`{array[2], array[3]}, {array[5], array[6]}`
read `array[8]` (miss)	`{array[5], array[6]}, {array[8], array[9]}`

C and cache misses (4)

```
typedef struct {
    int a_value, b_value;
    int other_values[6];
} item;
item items[5];
int a_sum = 0, b_sum = 0;
for (int i = 0; i < 5; ++i)
    a_sum += items[i].a_value;
for (int i = 0; i < 5; ++i)
    b_sum += items[i].b_value;
```

Assume everything but `items` is kept in registers (and the compiler does not do anything funny).

C and cache misses (4, rewrite)

```
int array[40]
int a_sum = 0, b_sum = 0;
for (int i = 0; i < 40; i += 8)
    a_sum += array[i];
for (int i = 1; i < 40; i += 8)
    b_sum += array[i];
```

Assume everything but `array` is kept in registers (and the compiler does not do anything funny) and array starts at beginning of cache block.

How many *data cache misses* on a *2-way* set associative 128B cache with 16B cache blocks and LRU replacement?

C and cache misses (4, solution pt 1)

ints 4 byte $\rightarrow$ array[0 to 3] and array[16 to 19] in same cache set

- 64B = 16 ints stored per way
- 4 sets total

accessing 0, 8, 16, 24, 32, 1, 9, 17, 25, 33

0 (set 0), 8 (set 2), 16 (set 0), 24 (set 2), 32 (set 0)

1 (set 0), 9 (set 2), 17 (set 0), 25 (set 2), 33 (set 0)

C and cache misses (4, solution pt 2)

access	set 0 after (LRU first)	result
—	—, —	
array[0]	—, array[0 to 3]	miss
array[16]	array[0 to 3], array[16 to 19]	miss
array[32]	array[16 to 19], array[32 to 35]	miss
array[1]	array[32 to 35], array[0 to 3]	miss
array[17]	array[0 to 3], array[16 to 19]	miss
array[32]	array[16 to 19], array[32 to 35]	miss

6 misses for set 0

C and cache misses (4, solution pt 3)

access	set 2 after (LRU first)	result
—	—, —	
array[8]	—, array[8 to 11]	miss
array[24]	array[8 to 11], array[24 to 27]	miss
array[9]	array[8 to 11], array[24 to 27]	hit
array[25]	array[16 to 19], array[32 to 35]	hit

2 misses for set 1

C and cache misses (struct)

```
typedef struct {
    int a_value, b_value;
    int other_values[10];
} item;
item items[5];
int a_sum = 0, b_sum = 0;
for (int i = 0; i < 5; ++i)
    a_sum += items[i].a_value;
for (int i = 0; i < 5; ++i)
    b_sum += items[i].b_value;
```

observation: 12 ints in struct: only first two used

equivalent to accessing array[0], array[12], array[24], etc.

… then accessing array[1], array[13], array[25], etc.

C and cache misses (struct, rewritten?)

```
typedef struct {
    int a_value, b_value;
    int other_values[10];
} item;
item items[5];
int a_sum = 0, b_sum = 0;
for (int i = 0; i < 5; ++i)
    a_sum += items[i].a_value;
for (int i = 0; i < 5; ++i)
    b_sum += items[i].b_value;
```

```
int array[60];
int a_sum = 0, b_sum = 0;
for (int i = 0; i < 60; i += 12)
    a_sum += array[i];
for (int i = 1; i < 60; i += 12)
    b_sum += array[i];
```

C and cache misses (struct, rewritten?)

```
int array[60];
int a_sum = 0, b_sum = 0;
for (int i = 0; i < 60; i += 12)
    a_sum += array[i];
for (int i = 1; i < 60; i += 12)
    b_sum += array[i];
```

Assume everything but `array` is kept in registers (and the compiler does not do anything funny) and array at beginning of cache block.

How many *data cache misses* on a 128B two-way set associative cache with 16B cache blocks and LRU replacement?

observation 1: first loop has 5 misses — first accesses to blocks

observation 2: array[0] and array[1], array[12] and array[13], etc. in same cache block

C and cache misses (struct, solution written)

ints 4 byte $\rightarrow$ array[0 to 3] and array[16 to 19] in same cache set

- 64B = 16 ints stored per way
- 4 sets total

accessing array indices 0, 12, 24, 36, 48, 1, 13, 25, 37, 49

0 (set 0, array[0 to 3]), 12 (set 3), 24 (set 2), 36 (set 1), 48 (set 0)

- each set used at most twice
- no replacement needed

so access to 1, 21, 41, 61, 81 all hits:

- set 0 contains block with array[0 to 3]
- set 5 contains block with array[20 to 23]
- etc.

C and cache misses (struct, rewritten?)

```
int array[60];
int a_sum = 0, b_sum = 0;
for (int i = 0; i < 60; i += 12)
    a_sum += array[i];
for (int i = 1; i < 60; i += 12)
    b_sum += array[i];
```

Assume everything but `array` is kept in registers (and the compiler does not do anything funny) and array at beginning of cache block.

How many *data cache misses* on a 64B direct-mapped cache with 16B cache blocks?

observation 1: first loop has 5 misses — first accesses to each block

observation 2: array[0] and array[1], array[12] and array[13], etc. in same cache block

C and cache misses (struct, solution setup 1)

array[0 to 3]	set 0
array[4 to 7]	set 1
array[8 to 11]	set 2
array[12 to 15]	set 3
array[16 to 19]	set 0
array[20 to 23]	set 1
array[20 to 23]	set 1
…	…

C and cache misses (struct, solution setup)

access	set	block loaded (if needed)
array[0]	0	array[0 to 3]
array[12]	3	array[12 to 15]
array[24]	2	array[24 to 27]
array[36]	1	array[36 to 39]
array[48]	0	array[48 to 51]
array[1]	0	array[0 to 3]
array[13]	3	array[12 to 15]
array[25]	2	array[24 to 27]
array[37]	1	array[36 to 39]
array[49]	0	array[48 to 51]

C and cache misses (struct, solution)

access	set	block loaded (if needed)	result
array[0]	0	array[0 to 3]	miss
array[12]	3	array[12 to 15]	miss
array[24]	2	array[24 to 27]	miss
array[36]	1	array[36 to 39]	miss
array[48]	0	array[48 to 51]	miss
array[1]	0	array[0 to 3]	miss*
array[13]	3	array[12 to 15]	hit
array[25]	2	array[24 to 27]	hit
array[37]	1	array[36 to 39]	hit
array[49]	0	array[48 to 51]	miss*

C and cache misses (struct, solution written)

ints 4 byte $\rightarrow$ array[0 to 3] and array[16 to 19] in same cache set

- 64B = 16 ints stored per way
- 4 sets total

accessing array indices 0, 12, 24, 36, 48, 1, 13, 25, 37, 49

- 0, 12, 24, 36, 48 all misses

0 (set 0, array[0 to 3]), 12 (set 3, 12-15), 24 (set 2, 24-27), 36 (set 1, 36-39), 48 (set 0, 48-51)

- 13, 25, 37 will be hits (rest of block already loaded)
- 1 will be a miss (0-3 was replaced with 48-51 when 48 accessed)
- 49 will be a miss (48-51 was replaced with 0-3 when 1 accessed)

C and cache misses (3)

```
typedef struct {
    int a_value, b_value;
    int boring_values[126];
} item;
item items[8]; // 4 KB array
int a_sum = 0, b_sum = 0;
for (int i = 0; i < 8; ++i)
    a_sum += items[i].a_value;
for (int i = 0; i < 8; ++i)
    b_sum += items[i].b_value;
```

Assume everything but `items` is kept in registers (and the compiler does not do anything funny).

How many *data cache misses* on a 2KB direct-mapped cache with 16B cache blocks?

C and cache misses (3, rewritten?)

```
item array[1024]; // 4 KB array
int a_sum = 0, b_sum = 0;
for (int i = 0; i < 1024; i += 128)
    a_sum += array[i];
for (int i = 1; i < 1024; i += 128)
    b_sum += array[i];
```

C and cache misses (4)

```
typedef struct {
    int a_value, b_value;
    int boring_values[126];
} item;
item items[8]; // 4 KB array
int a_sum = 0, b_sum = 0;
for (int i = 0; i < 8; ++i)
    a_sum += items[i].a_value;
for (int i = 0; i < 8; ++i)
    b_sum += items[i].b_value;
```

Assume everything but `items` is kept in registers (and the compiler does not do anything funny).

How many *data cache misses* on a 4-way set associative 2KB direct-mapped cache with 16B cache blocks?

thinking about cache storage (1)

2KB direct-mapped cache with 16B blocks —

set 0: address 0 to 15, (0 to 15) + 2KB, (0 to 15) + 4KB, …

- block at 0: array[0] through array[3]
- block at 0+2KB: array[512] through array[515]

set 1: address 16 to 31, (16 to 31) + 2KB, (16 to 31) + 4KB, …

- block at 16: array[4] through array[7]
- block at 16+2KB: array[516] through array[519]

…

set 127: address 2032 to 2047, (2032 to 2047) + 2KB, …

- block at 2032: array[508] through array[511]
- block at 2032+2KB: array[1020] through array[1023]

thinking about cache storage (1)

2KB direct-mapped cache with 16B blocks —

set 0: address 0 to 15, (0 to 15) + 2KB, (0 to 15) + 4KB, …

block at 0: *array[0] through array[3]*

block at 0+2KB: *array[512] through array[515]*

set 1: address 16 to 31, (16 to 31) + 2KB, (16 to 31) + 4KB, …

block at 16: array[4] through array[7]

block at 16+2KB: array[516] through array[519]

…

set 127: address 2032 to 2047, (2032 to 2047) + 2KB, …

block at 2032: array[508] through array[511]

block at 2032+2KB: array[1020] through array[1023]

thinking about cache storage (2)

2KB 2-way set associative cache with 16B blocks: block addresses —

set 0: address 0, 0 + 2KB, 0 + 4KB, …

- block at 0: array[0] through array[3]
- block at 0+1KB: array[256] through array[259]
- block at 0+2KB: array[512] through array[515]
- …

set 1: address 16, 16 + 2KB, 16 + 4KB, …

- address 16: array[4] through array[7]

…

set 63: address 1008, 2032 + 2KB, 2032 + 4KB …

- address 1008: array[252] through array[255]

thinking about cache storage (2)

2KB 2-way set associative cache with 16B blocks: block addresses —

set 0: address 0, 0 + 2KB, 0 + 4KB, …

block at 0: *array[0] through array[3]*

block at 0+1KB: *array[256] through array[259]*

block at 0+2KB: *array[512] through array[515]*

…

set 1: address 16, 16 + 2KB, 16 + 4KB, …

address 16: array[4] through array[7]

…

set 63: address 1008, 2032 + 2KB, 2032 + 4KB …

address 1008: array[252] through array[255]

arrays and cache misses (3)

```
int sum; int array[1024]; // 4KB array
for (int i = 8; i < 1016; i += 1) {
    int local_sum = 0;
    for (int j = i - 8; j < i + 8; j += 1) {
        local_sum += array[i] * (j - i);
    }
    sum += (local_sum - array[i]);
}
```

Assume everything but `array` is kept in registers (and the compiler does not do anything funny).

How many *data cache misses* on initially empty *2KB* direct-mapped cache with 16B cache blocks?

mapping arrays to cache sets

2KB direct-mapped cache w/ 16B blocks

$rightarrow$

values 2KB apart map to same set

- mapping wraps around to beginning of cache
- 2K = $2^{(textindexbits + textoffsetbits)}$ s

array[0–3] and array[512–515] map to same set

array[4–7] and array[516–519] map to same set

…

set-associative not better?

2KB direct-mapped cache, 4B array elements: array[0 to 3] and array[512 to 515] map to same set

- 2KB apart — mapping "wraps around"
- four misses for this set
- access to array[0]+array[2], then array[512]+array[514],
- then array[1]+array[3], then array[513]+array[515]
- same for each of other 127 sets

2KB, 2-way set associative cache: array[0 to 3] and array[256 to 259] and array[512 to 515] and array[756 to 759] map to same set

- 1KB apart — shorter time until mapping wraps around
- eight misses + same for other 63 sets

‘flat’ 2D arrays and cache blocks



